

Trabajo Practico

Programacion 1

Integrantes:

Maldonado Ariel Ramon - Nombre en GitHub: ArielDre-Boot

Franco Nehuen Gaude - Nombre en GitHub: lucabr2133

Luca Leandro Brandan - Nombre en GitHub: Nehuen0245

Introducción:

En el juego se controla a un personaje, el cual tiene que moverse por un mapa con múltiples plataformas, esquivando enemigos que vienen por derecha y por izquierda, además de un enemigo volando arriba del todo que lanzará proyectiles hacia el jugador de vez en cuando. Al final del mapa hay un castillo, si el personaje llega hasta el castillo entonces ganará y perderá si se le acaban las vidas.

El jugador podrá moverse a la derecha con la flecha derecha del teclado, a la izquierda con la flecha izquierda del teclado, saltar con la flecha hacia arriba del teclado, disparar un único proyectil con el click izquierdo que desaparece cuando choca con un enemigo o sale de pantalla y desplegar una barrera que dura unos segundo con el clik derecho.

Si el personaje choca con un enemigo perderá una vida y el enemigo desaparecerá, también perderá una vida cuando choque con el proyectil del enemigo volador. También si el personaje se cae entre las islas este también pierde una vida

Al ganar o perder saltará una pantalla que anuncia si ganó o perdió, apretando la tecla “R” el usuario podrá reiniciar el juego.

Descripción de las clases

Clase Juego

En la clase juego se utilizan diferentes variables y arrays.

```
private Entorno entorno; # Para utilizar métodos de la clase entorno  
private Personaje p; # Para mostrar y llamar a metodos del personaje
```

Tres arrays de la clase “Obstaculo”: obstaculosSuperiores, obstaculosInferiores, obstaculosBase. Utilizados para guardar las plataformas.

```
private Castillo castillo; # Para mostrar y utilizar los metodos del castillo
```

```
private Enemigo[] enemigos = new Enemigo[7]; # Para guardar los enemigos que aparecieran
```

```
private int enemigosVivos = 0; # Para contar cuantos enemigos quedan vivos  
private Enemigo2[] enemigos2 = new Enemigo2[1]; # Para guardar los enemigos voladores
```

```
private int enemigosVivos2 = 0; # Para contar cuandos enemigos voladores quedan vivos
```

private Escritor t; # Para mostrar por pantalla texto

private MostradorDeVida[] vidas = new MostradorDeVida[10]; # Para guardar cuantas las vidas

private Image imagenFondo; # Para utilizar como fondo mientras se esta jugando

private Image imagenFondoTermino = null; #Para utilizar como fondo cuando se muestre la pantalla de ganar o perder.

En el ciclo de **“Juego()”** se inicializan: entorno, p, castillo, la imagen de fondo del juego, el contenido de array vidas y t.

En el ciclo de **“tick()”** se añade un condicional, si “imagenFondoTermino” es null, entonces se sigue con el juego normalmente y si se cumple, salta a la pantalla de final.

Variables locales

bloqueaIzquierda: indica si el personaje no puede moverse hacia la izquierda debido a una colisión.

bloqueaDerecha: indica si el personaje no puede moverse hacia la derecha.

bloqueaArriba: indica si el personaje tiene un obstáculo encima.

bloqueaAbajo: indica si el personaje está apoyado sobre una superficie o colisionando desde abajo.

Se dibuja el fondo del juego centrado en la pantalla.

Se recorren las vidas del jugador y se dibujan si existen.

Se dibuja el personaje principal.

Detección de colisiones con obstáculos

El método analiza tres grupos de obstáculos:

Obstáculos superiores (**obstaculosSuperiores**)

- Se recorren todos los obstáculos de este grupo.
- Cada obstáculo se dibuja en pantalla.
- Se verifica si el personaje colisiona con ellos desde:
 - izquierda
 - derecha
 - arriba
 - abajo
- Si hay colisión, se activan las variables de bloqueo correspondientes.

Obstáculos inferiores (**obstaculosInferiores**)

- Se realiza el mismo procedimiento que con los superiores.
- Permiten detectar colisiones en otra zona del mapa (niveles más bajos).

Obstáculos de la base (**obstaculosBase**)

- Representan el suelo o base del escenario.
- También se dibujan y se evalúan colisiones en todas las direcciones.

Condición de generación de enemigos

Se crea un enemigo únicamente si:

- La posición del arreglo está vacía (**null**).
- La cantidad de enemigos vivos es menor al máximo permitido.
- El jugador no ganó ni perdió la partida.

Generación de valores aleatorios

Se utilizan valores aleatorios para definir:

- El lado desde donde aparece el enemigo (izquierda o derecha).
- La posición vertical en la que aparece.
- La distancia inicial fuera de la pantalla.

Creación del enemigo

- Si aparece desde la derecha, se crea fuera de la pantalla y se mueve hacia la izquierda.
- Si aparece desde la izquierda, se crea fuera de la pantalla y se mueve hacia la derecha.

Luego:

- Se guarda en el arreglo **enemigos**.
- Se incrementa el contador de enemigos vivos.

Generación de enemigos (**Enemigo2**)

Se recorre el arreglo de enemigos de tipo **Enemigo2**.

Condición de generación

Se crea un enemigo únicamente si:

- La posición del arreglo está vacía.
- No se alcanzó el máximo de enemigos de este tipo.
- El jugador no ganó ni perdió.

Generación de valores aleatorios

Se generan valores para:

- La dirección de movimiento (izquierda o derecha).
- La posición vertical inicial (fuera de la pantalla, por arriba).
- La posición horizontal dentro del ancho de la pantalla.

Creación del enemigo

- El enemigo aparece por encima de la pantalla (posición negativa en Y).
- Se le asigna una dirección de movimiento horizontal.

Luego:

- Se guarda en el arreglo `enemigos2`.
- Se incrementa el contador de enemigos vivos de este tipo.

Recorrido de enemigos2

Se recorre el arreglo `enemigos2`:

- Se obtiene cada enemigo en la posición actual.
- Se verifica que el enemigo exista y que el jugador no haya ganado ni perdido.

Colisión con disparo del jugador

Se verifica si el jugador tiene un disparo activo y si este impacta al enemigo:

- Si hay colisión:
 - Se reproduce un sonido de explosión.
 - El enemigo se elimina del arreglo (`null`).
 - Se elimina el disparo del jugador.
 - Se decrementa el contador de enemigos vivos.
 - Se pasa al siguiente enemigo (`continue`).

Colisión con barrera

Se verifica si el enemigo colisiona con la barrera del jugador:

- Si hay colisión:
 - El enemigo se elimina.
 - Se reduce el contador de enemigos vivos.
 - Se reproduce un sonido de explosión.
 - Se continúa con el siguiente enemigo.

Dibujo del enemigo

Si el enemigo sigue activo:

- Se dibuja en pantalla utilizando el método `dibujar`.

Colisión con el jugador

Se verifica si el enemigo impacta directamente al jugador:

- Si ocurre:
 - El enemigo se elimina.
 - Se decrementa el contador de enemigos vivos.
 - Se actualizan las vidas del jugador mediante `convertirANUUILaVida`.
 - Se continúa con el siguiente enemigo.

Movimiento del enemigo

El comportamiento depende de su posición vertical:

Descenso inicial

- Si el enemigo está cerca de la parte superior ($y \leq 50$):
 - Se mueve hacia abajo.

Comportamiento en juego

Cuando ya ingresó a la zona jugable:

Disparo

- Si no tiene un disparo activo:
 - Se reproduce un sonido de disparo.
 - Se genera un disparo dirigido al jugador.
 - Se reinicia el estado de impacto del disparo.
- Si tiene un disparo:
 - Se dibuja el proyectil.
 - Se mueve el proyectil.
 - Se verifica si impacta al jugador:
 - Si impacta:
 - Se reproduce un sonido de explosión.
 - Se reducen las vidas del jugador.

Movimiento horizontal

- Si la dirección es "derecha":
 - Se mueve hacia la derecha.
 - Si llega al límite, cambia dirección a "izquierda".
- Si la dirección es "izquierda":
 - Se mueve hacia la izquierda.
 - Si llega al límite, cambia dirección a "derecha".

Recorrido de enemigos

Se recorre el arreglo `enemigos`:

- Se obtiene cada enemigo en la posición actual.
- Se verifica que el enemigo exista ($!= \text{null}$) antes de procesarlo.

Colisión con disparo del jugador

Se verifica si el jugador tiene un disparo activo:

- Si el disparo impacta al enemigo:
 - El enemigo se elimina (`null`).
 - Se elimina el disparo del jugador.
 - Se decrementa el contador de enemigos vivos.
 - Se reproduce un sonido de explosión.

Colisión con obstáculos

Se recorren los obstáculos superiores e inferiores:

- Si el enemigo colisiona con un obstáculo:
 - El enemigo se elimina.
 - Se decrementa el contador de enemigos vivos.

Esto evita que los enemigos atraviesen estructuras del escenario.

Colisión con barrera

Se verifica si el enemigo colisiona con la barrera del jugador:

- Si ocurre:
 - El enemigo se elimina.
 - Se decrementa el contador de enemigos vivos.
 - Se reproduce un sonido de explosión.

Dibujo del enemigo

Si el enemigo sigue existiendo:

- Se dibuja en pantalla con el método `dibujar`.

Colisión con el jugador

Se verifica si el enemigo impacta directamente al jugador:

- Si hay colisión:
 - El enemigo se elimina.
 - Se decrementa el contador de enemigos vivos.
 - Se reproduce un sonido de explosión.
 - Se reducen las vidas del jugador mediante `convertirANUILLaVida`.

Movimiento del enemigo

Si no hubo colisión con el jugador:

Movimiento hacia la izquierda

- Si la dirección es `"izquierda"`:
 - El enemigo se mueve hacia la izquierda.
 - Si sale de la pantalla:
 - Se elimina el enemigo.
 - Se decrementa el contador de enemigos vivos.

Movimiento hacia la derecha

- Si la dirección es `"derecha"`:
 - El enemigo se mueve hacia la derecha.
 - Si sale de la pantalla:
 - Se elimina el enemigo.
 - Se decrementa el contador de enemigos vivos.

Generación de plataformas

Línea de obstáculos superiores (`obstaculosSuperiores`)

Inicialización de valores

- **valorY**: posición vertical fija (400).
- **anchoObstaculo** y **altoObstaculo**: dimensiones de cada obstáculo.

Generación de obstáculos

Se recorre el arreglo:

- Si la posición está vacía y el jugador **no se está moviendo**:
 - Se genera un obstáculo en una posición X calculada con:
 - Un desplazamiento fijo ($500 * i$).
 - Un valor aleatorio para variar la ubicación.

Movimiento

- Si el obstáculo existe y el jugador **está en movimiento**:
 - Se desplaza hacia la izquierda (-2 en X).

Esto genera el efecto de que el escenario se mueve.

Dibujo

- Cada obstáculo se dibuja en pantalla en cada iteración.

Línea de obstáculos inferiores (obstaculosInferiores)

Funciona de manera similar a la línea superior, con algunas diferencias:

Características

- Posición vertical fija distinta (**valorY1 = 500**).
- Diferente cálculo de separación horizontal ($400 * i$).

Comportamiento

- Se generan obstáculos cuando no hay movimiento.
- Se desplazan hacia la izquierda cuando el jugador se mueve.
- Se dibujan en cada ciclo.

Línea de obstáculos base (obstaculosBase)

Representa el suelo del juego.

Generación

- Si la posición está vacía y el jugador no se mueve:
 - Se crean obstáculos alineados horizontalmente ($400 * i$).
 - Tienen mayor tamaño (simulan el piso).

Movimiento

- Si el jugador está en movimiento:
 - Los obstáculos se desplazan hacia la izquierda.

Control de caída del personaje

Se verifica si el personaje toca el borde inferior de la pantalla:

- Se utiliza el método `siElPersonajeTocaElBordeInferiorDeLaPantalla`.

Si la condición se cumple:

- Se actualizan las vidas del jugador mediante `convertirANUILLaVida`.
- Este método:
 - Resta una vida.
 - Actualiza la interfaz de vidas.
 - Reposiciona o reinicia al personaje.

Si gano o perdio

Se verifica si el jugador ganó o perdió:

- `p.isGano()` indica que el jugador cumplió el objetivo del juego.
- `p.isPerdio()` indica que el jugador se quedó sin vidas

Si cualquiera de las dos condiciones es verdadera, el juego se considera finalizado.

- Se carga una imagen desde los recursos del juego utilizando `Herramientas.cargarImagen`.
- La imagen cargada (`Corazon.jpg`) se asigna a la variable `imagenFondoTermino`.

Pantalla de fin de juego y reinicio

Este bloque de código se ejecuta cuando el juego ya terminó (es decir, cuando existe `imagenFondoTermino`) y se encarga de mostrar la pantalla final y permitir reiniciar la partida.

Dibujo del fondo final

- Se dibuja la imagen almacenada en `imagenFondoTermino`.
- Se posiciona en el centro de la pantalla.
- Se aplica una escala menor (0.08) para ajustarla visualmente.

Indicador de resultado

Se muestra un mensaje según el estado del jugador:

- Si el jugador perdió (`p.isPerdio()`):
 - Se dibuja la letra 'L' (Lose).
- Si el jugador ganó (`p.isGano()`):
 - Se dibuja la letra 'W' (Win).

Esto permite identificar claramente el resultado de la partida.

Reinicio del juego

Se detecta si el jugador presiona la tecla **R**:

- Si se presiona 'r' o 'R', se reinicia la partida.

Acciones al reiniciar

- Se restauran las vidas mediante `sumarVida`.
- Se eliminan todos los enemigos usando `eliminarTodosLosEnemigos`.
- Se actualiza el contador de enemigos vivos.
- Se reposiciona al personaje en su punto inicial (`x=140, y=300`).
- Se elimina la imagen de fondo final (`imagenFondoTermino = null`).
- Se reinician los estados del jugador:
 - `p.setGano(false)`
 - `p.setPerdio(false)`

Funciones

detectaElMovimiento(Entorno a, Personaje b)

- Si se presiona la tecla derecha:
 - Y el personaje está en la mitad derecha de la pantalla (`b.getX() >= 400`)
→ entonces el personaje entra en estado de movimiento (`setEnMovimiento(true)`).
- En cualquier otro caso:
→ el personaje no se mueve (`setEnMovimiento(false)`).
- Si se presionan **derecha e izquierda al mismo tiempo**:
→ el movimiento se cancela (`setEnMovimiento(false)`).

controlDelProyectil(Personaje p, Entorno entorno)

Dibujar el proyectil

- Si el personaje tiene un disparo (`getDisparo() != null`):
→ se dibuja en pantalla.

Movimiento del proyectil

- Si existe:
→ se actualiza su posición con `mover()`.

Colisiones con los bordes

El proyectil se elimina (`setDisparo(null)`) si sale de la pantalla:

- **Izquierda:** `x < 0`
- **Derecha:** `x > entorno.ancho()`
- **Arriba:** `y < 0`
- **Abajo:** `y > entorno.alto()`

controlDelProyectilEnemigo(Enemigo2 e, Entorno entorno)

Primero se verifica que el enemigo tenga un disparo:

- `e.getDisparo() != null`

Luego se controlan las colisiones con los bordes:

- **Izquierda:**
 - Si `x < 0` → se elimina el disparo.
- **Derecha:**
 - Si `x > entorno.ancho()` → se elimina.
- **Arriba:**
 - Si `y < 0` → se elimina.
- **Abajo:**
 - Si `y > entorno.alto()` → se elimina.

controlDelBarrera(Entorno e, Personaje p)

Dibujar y actualizar la barrera

Si el personaje tiene una barrera:

- Se dibuja en pantalla.
- Se mueve junto al personaje (`movimiento`).
- Se actualiza el tiempo actual usando `e.tiempo()`.

Control de duración

- Se verifica si el tiempo actual superó el tiempo de desaparición:

`getTiempoActual() >= getTiempoDeDesaparicion()`

- Si esto ocurre:
 - la barrera se elimina (`setBarrera(null)`).

controlMovimientoJugador(Entorno entorno, Personaje p, boolean bloqueaIzquierda, boolean bloqueaDerecha, boolean bloqueaAbajo, boolean bloqueaArriba)

Parámetros de bloqueo

- **bloqueaIzquierda:** impide moverse a la izquierda.
- **bloqueaDerecha:** impide moverse a la derecha.
- **bloqueaAbajo:** indica si está apoyado en una superficie.
- **bloqueaArriba:** indica si hay algo encima.

Salto

if (TECLA_ARRIBA && !bloqueaArriba && (bloqueaAbajo || toca el piso))

- Solo puede saltar si:
 - No tiene algo arriba.
 - Está apoyado en el piso o en una plataforma.
 - Al saltar:
 - Se activa **estaSaltando**.
 - Se define hasta dónde va a subir (**saltoY**).
-

2. Movimiento a la izquierda

- Si se presiona izquierda:
 - No está bloqueado.
 - No sale de la pantalla.
 - se mueve hacia la izquierda.
 - Si no:
 - se desactiva el estado (**setAtras(false)**).
-

3. Movimiento a la derecha

- Si se presiona derecha:
 - No está bloqueado.
 - No sale de la pantalla.
 - se mueve hacia la derecha.
 - Si no:
 - se desactiva (**setAdelante(false)**).
-

4. Gravedad

if (tieneGravedad && no toca el piso && !bloqueaAbajo)

- El personaje cae automáticamente.

Disparo

- Click izquierdo:
 - Si no hay disparo activo → dispara.
- Click derecho:
 - Si no hay disparo → crea la barrera.

controlDelSalto(Personaje p, boolean bloqueaArriba)

Si el personaje está saltando:

Subida del salto

- Si no hay nada arriba:
 - Se desactiva la gravedad.
 - El personaje sube (**saltoY**).
 - Cuando llega a la altura máxima (**saltoY**):
 - Se vuelve a activar la gravedad.
 - Se termina el salto.
-

Colisión con techo

- Si hay algo arriba (**bloqueaArriba**):
 - Se corta el salto automáticamente.
 - Se reactiva la gravedad.

convertirANULLaVida(Entornoe, mostradorDeVida[] v, Personaje p)

Recorre el arreglo de vidas **desde el final hacia el inicio**.

- Busca la última vida disponible (**!= null**):
 - La elimina (**v[i] = null**).
 - Usa una bandera (**yaSeQuito**) para asegurarse de borrar **solo una**.
-

Condición de derrota

- Si la primera posición (**v[0]**) queda en **null**:
 - Se reproduce un sonido (**lose.wav**).
 - El personaje pasa a estado de derrota (**setPerdio(true)**).

sumarVida(Entorno e, MostradorDeVida[] v, Personaje p, int vidaSumada)

Primer if: Reinicio (vidaSumada == -1)

- Se recrean todas las vidas desde cero:
 - Se recorre hasta `p.getVida()`.
 - Se crean nuevos objetos `MostradorDeVida`.
 - Se posicionan en pantalla (`i * 50 + 50`).

👉 Se usa cuando el juego se reinicia.

Segundo if: Sumar vida (vidaSumada > 0)

- Busca espacios vacíos (`null`) en el arreglo.
- Agrega nuevas vidas en esas posiciones.

eliminarTodosLosEnemigos(Entorno e, Enemigo[] enemigos)

Recorre todo el arreglo de enemigos.

Si encuentra uno distinto de `null`:

- Lo elimina (`enemigos[i] = null`).

Clase “Barrera”

Variables de instancia

x: coordenada horizontal de la barrera.

y: coordenada vertical de la barrera.

ancho: ancho de la barrera.

distancia: altura de la barrera.

tiempoActual: instante en el que la barrera fue creada o actualizada.

tiempoDeDesaparicion: momento en el que la barrera debería desaparecer.

Constructor

Barrera(double x, double y, double ancho, double distancia, int tiempo)

- Inicializa la posición y tamaño de la barrera.
- Guarda el tiempo actual.
- Define un tiempo de desaparición sumando 1000 unidades al tiempo inicial.

Métodos principales

dibujar(Entorno e):

- Actualiza el tiempo de aparición de la barrera.
- Dibuja un rectángulo rojo en pantalla representando la barrera.
- Se posiciona con un pequeño desplazamiento en X ($x + 20$).

actualizarTiempoDeAparicion(Entorno e, double tiempo):

- Actualiza el valor de **tiempoActual**.
- Permite llevar control del tiempo de vida de la barrera.

movimiento(int xDePersonaje, int yDePersonaje):

- Hace que la barrera siga la posición del personaje.
- Actualiza sus coordenadas en base al jugador.

Métodos de bordes (colisiones)

Estos métodos devuelven los límites de la barrera:

- **bordeDerecho()**: posición del lado derecho.
- **bordeIzquierdo()**: posición del lado izquierdo.
- **bordeInferior()**: posición de la parte inferior.
- **bordeSuperior()**: posición de la parte superior.

Problemas encontrados y soluciones

El principal problema que surgió es que algunos los enemigos eran demasiado rápidos, entonces pasaban la barrera y chocaban con el jugador, esto se solucionó moviendo la hitbox de la barrera más adelante, aunque no correspondiera con lo que se muestra por pantalla. Después estuvo el tiempo de aparición, que se tuvo que ajustar a ojo.

Clase “MostradorDeVida”

Variables de instancia

x: coordenada horizontal donde se dibuja la vida.

y: coordenada vertical donde se dibuja la vida.

imagen: imagen que representa la vida (ícono de corazón).

Constructor

MostradorDeVida(int x, int y)

- Inicializa la posición donde se mostrará la vida.
- Carga una imagen desde los recursos del juego (**Corazon.jpg**).
- Guarda esa imagen en la variable **imagen**.

Métodos principales

dibujar(Entorno e):

- Dibuja la imagen del corazón en pantalla.
- Se posiciona en las coordenadas (**x, y**).

- Se aplica una escala (0.08) para ajustar su tamaño.

Problemas encontrados y soluciones

Un problema que surgió fue que no se cargaba la imagen, esto se solucionó pasando a las imágenes a .jpg, después estaba el tamaño de las imágenes, que eran demasiado grandes, se solucionó ajustando su tamaño a “0.08”.

Clase “Escritor”

Variables de instancia

x: coordenada horizontal donde se mostrará el texto.

y: coordenada vertical donde se mostrará el texto.

textoGanador: mensaje que se mostrará cuando el jugador gane.

textoPerdedor: mensaje que se mostrará cuando el jugador pierda.

Constructor

Escritor(int x, int y, String textoGanador, String textoPerdedor)

- Inicializa la posición donde se dibujará el texto.
- Guarda los mensajes de victoria y derrota.

Métodos principales

dibujar(char c, Entorno e):

- Configura la fuente del texto:
 - Tipo: Arial
 - Tamaño: 50
 - Color: blanco
- Según el valor del parámetro **c**:
 - Si es 'W':
 - Se muestra el texto de victoria (**textoGanador**).
 - Si es 'L':
 - Se muestra el texto de derrota (**textoPerdedor**).

Problemas encontrados y soluciones

surgió el problema de que teníamos que encontrar un color para que sea fácilmente visible, se decidió dejar de mostrar la pantalla del juego y en su lugar mostrar una pantalla en negro con una imagen y abajo poner las letras.

Clase Enemigo

La clase `Enemigo` representa un enemigo básico del juego. Su función es desplazarse horizontalmente por la pantalla, interactuar con el jugador y detectar colisiones con distintos elementos del escenario, como obstáculos y barreras.

Variables de instancia

- `x`: coordenada horizontal del enemigo.
- `y`: coordenada vertical del enemigo.
- `ancho`: ancho del enemigo utilizado para dibujar y calcular colisiones.
- `alto`: altura del enemigo utilizada para dibujarlo y calcular colisiones.
- `velocidadX`: velocidad de desplazamiento horizontal del enemigo.
- `direccion`: indica la dirección actual de movimiento ("izquierda" o "derecha").

Métodos principales

- `moverIzquierda()`: desplaza el enemigo hacia la izquierda.
- `moverDerecha()`: desplaza el enemigo hacia la derecha.
- `dibujar(Entorno e)`: dibuja el enemigo como un rectángulo azul en la pantalla.
- `esDestruibleDerecha(Entorno e)`: determina si el enemigo salió por el borde derecho de la pantalla.
- `esDestruiblePorIzquierda(Entorno e)`: determina si el enemigo salió por el borde izquierdo de la pantalla.
- `bordeIzquierdo()`: devuelve la coordenada del borde izquierdo.
- `bordeDerecho()`: devuelve la coordenada del borde derecho.
- `bordeSuperior()`: devuelve la coordenada del borde superior.
- `bordeInferior()`: devuelve la coordenada del borde inferior.
- `colisionaConElJugador(Personaje p)`: verifica si el enemigo impacta contra el jugador.
- `colisionaConObstaculo(Obstaculo o)`: verifica si el enemigo colisionó con un obstáculo.
- `colisionaConBarrera(Personaje p)`: verifica si el enemigo colisiona con la barrera creada por el jugador.

La clase incluye métodos `get` y `set` para acceder y modificar sus atributos.

Problemas encontrados y soluciones

Desplazamiento y colocación de los enemigos

Uno de los problemas fue el como hacer el movimiento de los enemigos, porque tenían que moverse de izquierda a derecha y viceversa, también como hacer para colocarlos correctamente, y destruirlos una vez fuera de la pantalla

La solución para el movimiento fue tener una variable de instancia que determina la dirección y según su dirección se destruye de un lado otro

Detección de colisiones entre los enemigos y su entorno

Uno de los primeros problemas surgidos, fue que los enemigos se generaban entre los niveles de las islas, otro fue que a cada enemigo se le tenía que chequear cada uno de los obstáculos

para solucionar el primero de los problemas, fue necesario detectar si un enemigo estaba en el mismo nivel que las islas, de ser así entonces el enemigo se vuelve nulo , el otro es hacer un bucle dentro del bucle de los enemigos y chequear si colisiona con este

Generación de enemigos infinitamente

Otro problema encontrado , fue el de cómo poder generar enemigos infinitamente usando una lista finita de elementos,

La solución fue crear un while en tick que solo se ejecutaba si la variable de enemigos actuales era menor al length del array , dentro se creaban numero random que determinaban la posiciones aleatorias del enemigo

Clase Enemigo 2

La clase **Enemigo 2** representa un enemigo del juego que puede desplazarse horizontal y verticalmente, detectar colisiones y disparar proyectiles hacia el jugador.

Variables de instancia

- x, y: almacenan la posición del enemigo en la pantalla.
- ancho, alto: representan las dimensiones del enemigo.
- velocidadX: velocidad de movimiento horizontal.
- velocidadY: velocidad de movimiento vertical.
- direccion: indica si el enemigo se mueve hacia la izquierda o hacia la derecha.
- aceleracion: valor utilizado para incrementar la velocidad vertical durante la caída.
- disparo: referencia al proyectil disparado por el enemigo.
- disparoTocoJugador: indica si el proyectil ya impactó al jugador.

Métodos

- dibujar(Entorno e): dibuja el enemigo en pantalla.

- `moverIzquierda()`: desplaza el enemigo hacia la izquierda.
- `moverDerecha()`: desplaza el enemigo hacia la derecha.
- `moverAbajo()`: mueve el enemigo hacia abajo aplicando aceleración.
- `cambiarDireccionDerecha(Entorno e)`: verifica si alcanzó el borde derecho de la pantalla.
- `cambiarDireccionIzquierda(Entorno e)`: verifica si alcanzó el borde izquierdo de la pantalla.
- `colisionaConElJugador(Personaje p)`: detecta colisiones entre el enemigo y el jugador.
- `colisionaConObstaculo(Obstaculo o)`: detecta colisiones con obstáculos.
- `colisionaConBarrera(Personaje p)`: detecta colisiones con la barrera del jugador.
- `disparo(Personaje p)`: genera un proyectil dirigido hacia la posición actual del jugador.
- `disparoColisionaJugador(Personaje p)`: verifica si el proyectil impactó al jugador.
- Métodos `get` y `set`: permiten acceder y modificar los atributos de la clase.

Problemas encontrados

Implementación de colisiones del disparo

El principal problema fue el hecho de detectar cuando el disparo colisionaba con el jugador, también hubo problemas en la generación de los enemigos 2 por que se tuvo que separar totalmente de enemigos 1

Para la detección del disparo se usó los métodos previamente conocidos de las otras clases, y siguiendo de base lo hecho con el disparo del jugador se pudo solucionar, para la generación se implementó una lista aparte de este tipo de enemigos con sus propios bucles que los controlan y generan

Disparos enemigos

Un problema menor fue el hecho de saber cuando dispara de nuevo un enemigo, cuando el enemigo disparaba solo lo podía hacer una vez

Se optó por así como el jugador su disparo desaparece cuando este mismo se va de los límites de pantalla, también el enemigo puede disparar cuando su disparo se va de los límites de pantalla

Gestión de vidas

Cuando el jugador chocaba con el proyectil, en un principio la colisión al hacerse en cada tick, hacía que el jugador perdiera muchas vidas, por que se seguía detectando

la pérdida de vidas en cada tick hasta que el proyectil haya salido del área del jugador

Se implementó un condicional que funciona cuando al jugador no se lo golpeo con el proyectil, y cuando se lo golpeo el condicional no vuelve a ocurrir

Clase Castillo

La clase Castillo representa la estructura principal que debe ser protegida durante la partida. Permanece fija en el escenario y puede ser utilizada para detectar colisiones con otros objetos del juego.

Variables de instancia

- x: posición horizontal del castillo.
- y: posición vertical del castillo.
- ancho: ancho del castillo.
- alto: alto del castillo.

Métodos principales

- Castillo(): constructor encargado de inicializar la posición y dimensiones del castillo.
- dibujar(): dibuja el castillo en pantalla mediante un rectángulo.
- bordeIzquierdo(), bordeDerecho(), bordeSuperior() y bordeInferior(): calculan los límites del castillo para la detección de colisiones.
- Métodos getters y setters: permiten consultar y modificar los atributos de la clase.

Problemas encontrados y soluciones implementadas

El problema principal fue saber cómo terminar el juego, cuando el jugador tocaba el castillo dado que las acciones como el desplazamiento de las islas seguían funcionando

Para solucionarlo se usó una variable dentro de jugador que mientras está fuera false las acciones transcurrían normalmente, pero cuando se ganaba se volvía true

Clase Personaje

Descripción general

La clase **Personaje** representa al jugador principal del juego. Es la entidad

controlada por el usuario y se encarga de gestionar el movimiento, los saltos, la gravedad, los disparos, la creación de barreras y las interacciones con el resto de los elementos del escenario. Además, almacena información relacionada con el estado de la partida, como la victoria o derrota del jugador.

Variables de instancia

- **x**: posición horizontal del personaje.
- **y**: posición vertical del personaje.
- **ancho**: ancho del personaje.
- **alto**: alto del personaje.
- **saltoY**: variable utilizada para controlar la lógica de los saltos.
- **disparo**: proyectil disparado por el personaje.
- **tieneGravedad**: indica si el personaje está siendo afectado por la gravedad.
- **estaSaltando**: indica si el personaje se encuentra realizando un salto.
- **seMueve**: indica si el personaje se está desplazando.
- **velocidadX**: velocidad de movimiento horizontal.
- **velocidadY**: velocidad de movimiento vertical.
- **vida**: cantidad de vidas o puntos de vida del personaje.
- **llegoAlCastillo**: indica si el personaje alcanzó el castillo.
- **gano**: indica si el jugador ganó la partida.
- **perdio**: indica si el jugador perdió la partida.
- **barrera**: referencia a la barrera creada por el personaje.

Métodos principales

- **moverIzquierda()**: desplaza el personaje hacia la izquierda.
- **moverDerecha()**: desplaza el personaje hacia la derecha.
- **moverArriba()**: mueve el personaje hacia arriba.
- **moverAbajo()**: mueve el personaje hacia abajo.
- **saltar()**: ejecuta la lógica de salto del personaje.
- **dibujar()**: dibuja el personaje en pantalla mediante un rectángulo de color rojo.
- **colisionaPorIzquierda()**: detecta colisiones con obstáculos desde la izquierda.
- **colisionaPorDerecha()**: detecta colisiones con obstáculos desde la derecha.
- **colisionaPorArriba()**: detecta colisiones con obstáculos desde arriba.
- **colisionaPorDebajo()**: detecta colisiones con obstáculos desde abajo.
- **colisionaCastillo()**: verifica si el personaje alcanzó el castillo.
- **rebote()**: desplaza al personaje para evitar que quede atrapado dentro de un obstáculo.
- **siElPersonajeTocaElBordeInferiorDeLaPantalla()**: reposiciona al personaje cuando cae fuera del escenario.
- **disparo()**: genera un proyectil dirigido hacia la posición actual del mouse.
- **creacionDeRayo()**: crea una barrera temporal para proteger al personaje.
- **bordeIzquierdo()**
- **bordeDerecho()**
- **bordeSuperior()**
- **bordeInferior()**

Métodos Getters y Setters

Permiten acceder y modificar los atributos de la clase.

Problemas encontrados y soluciones

Uno de los problemas principales fue la detección de colisiones contra los obstáculos, hubo muchos errores, se podría decir que fue lo que mas nos costo hacer bien, primeramente el jugador se quedaba atascado cuando chocaba con un obstáculo, para solucionar esto añadimos un rebote que hace que el jugador se mueve para atrás, evitando este error,

Otro de los errores fue que al haber múltiples obstáculos, el personaje se movía muy rápido y las colisiones funcionaban mal

Esto pasaba por que se ejecutaba el método varias veces y ahí se controlaba cosas como la vellosidad, por ende esto ocurría dos veces

Las solucion fue sacar del método la parte de la colisiones con los obstáculos, y hacer un for que chequea si un obstáculo colisionó con un jugador, de ser así pone un true en una variable correspondiente a cada uno de los lados, así se evitaba esto

Otro problema fue el movimiento de la cámara y de los obstáculos cuando el jugador se movía hasta el medio, esto produjo muchos errores, por ejemplo las colisiones no se detectaban bien dado que al cambiar de posición los obstáculos, a veces el desplazamiento del jugador se hacía cuando ya pasaba los bordes, también con el salto del jugador traspasaba los obstáculos, todo esto se soluciono ajustando las colisiones

Clase Proyectoil

Descripción general

La clase **Proyectoil** representa los disparos realizados tanto por el jugador como por algunos enemigos. Su función principal es desplazarse en línea recta hacia un objetivo determinado y detectar colisiones con otros objetos del juego.

Variables de instancia

- **x**: posición horizontal del proyectil.
- **y**: posición vertical del proyectil.
- **diametro**: tamaño del proyectil.
- **dx**: componente horizontal de la dirección de movimiento.
- **dy**: componente vertical de la dirección de movimiento.
- **velocidad**: velocidad constante del proyectil.

Métodos principales

- **Proyectoil(double x, double y, double diametro, int xMouse, int yMouse)** Inicializa la posición y el tamaño del proyectil. Además, calcula automáticamente la dirección hacia el punto de destino utilizando las coordenadas recibidas como parámetro.
- **dibujar()** Dibuja el proyectil en pantalla mediante un círculo de color azul.
- **mover()** Actualiza la posición del proyectil utilizando los valores de dirección calculados previamente.

- `colisionaDisparoConEnemigo()` Verifica si el proyectil impacta contra un objeto de tipo `Enemigo`.
- `colisionaDisparoConEnemigo2()` Verifica si el proyectil impacta contra un objeto de tipo `Enemigo2`.
- `bordeIzquierdo()`
- `bordeDerecho()`
- `bordeSuperior()`
- `bordeInferior()`

Permiten calcular los límites del proyectil para realizar verificaciones de colisión.

Métodos Getters y Setters

Permiten consultar y modificar los atributos del proyectil.

Problemas encontrados y soluciones

Los principales problemas fueron el cómo calcular la dirección y la distancia entre el mouse y el personaje, eso se pudo resolver usando lo visto en clase, con el teorema de pitágoras para poder saber el ángulo a donde el disparo tiene que ir

Otro tema fue el de la destrucción del proyectil cuando se iba fuera de la pantalla del juego , pero se soluciono detectando cuando estaba fuera y convirtiéndolo en null

Clase “Obstaculo”

Variables de instancia

x: coordenada horizontal del obstáculo.

y: coordenada vertical del obstáculo.

ancho: ancho del obstáculo.

alto: altura del obstáculo.

imagen: imagen que se utiliza para representar visualmente el obstáculo.

Constructor

`Obstaculo(int x, int y, int ancho, int alto)`

- Inicializa la posición (`x`, `y`).
 - Define sus dimensiones (`ancho`, `alto`).
 - Carga una imagen (`baseVerde1.png`) que será usada para dibujarlo.
-

Métodos principales

`dibujar(Entorno e):`

- Dibuja el obstáculo en pantalla usando la imagen.

- Aplica una escala de 0.190 (más chico).
-

dibujarBase(Entorno e):

- Dibuja el obstáculo con un tamaño más grande.
 - Usa una escala de 0.43.
 - Se utiliza principalmente para las bases del nivel.
-

Métodos de bordes (colisiones)

Estos métodos permiten detectar colisiones con otros objetos (como el personaje):

- **bordeDerecho()**: devuelve el límite derecho del obstáculo.
- **bordeIzquierdo()**: devuelve el límite izquierdo.
- **bordeInferior()**: devuelve la parte inferior.
- **bordeSuperior()**: devuelve la parte superior.

Problemas encontrados y soluciones

Otro problema que tuvimos fue el movimiento de los niveles y las bases, debido a que en un comienzo se optó por tener una cantidad fija de obstáculos en pantalla que al avanzar y toparse con el borde izquierdo de la pantalla, se modificaba su coordenada X y se teletransportaba automáticamente al lado derecho por fuera de la pantalla, para dar un efecto de reaparición de un nuevo nivel por ese lado, cuando en realidad simplemente era una cantidad fija y repetida de obstáculos que se teletransportaban. Para solucionar esto, creamos un arreglo de tipo "Obstáculo" con una longitud fija que luego lo recorrimos con un "for" desde la posición 0, hasta la posición anterior a la longitud del arreglo trabajado. Si la posición del arreglo era nula y el personaje no se movía, se creaba un nuevo objeto de tipo Obstáculo el cual era asignado a ese índice del arreglo el cual era nulo. Si no era nulo y el personaje se movía, significaba que existía un objeto en esa posición del arreglo, por lo cual solo se modificó su coordenada X dándole un valor negativo para que cada índice que cumpla la condición, se mueva hacia la izquierda. Una vez que cada obstáculo chocaba con el borde izquierdo, no desaparecía ni se teletransportaba, sino que su movimiento seguía hasta coordenadas negativas y nunca se convertía en nulo. Luego, por cada ciclo del arreglo, se llamaba a el método dibujar dentro de la clase "Obstáculo", que nos dibujaba la imagen elegida por sobre el obstáculo

Implementacion

Juego.java

```
package juego;

import java.util.Random;
import java.awt.Color;
import java.awt.Image;
import entorno.Entorno;
import entorno.Herramientas;
import entorno.InterfaceJuego;

public class Juego extends InterfaceJuego {
    // El objeto Entorno que controla el tiempo y otros
    private Entorno entorno;
    private Personaje p;

    private Obstaculo[] obstaculosSuperiores = new Obstaculo[8];
    private Obstaculo[] obstaculosInferiores = new Obstaculo[9];
    private Obstaculo[] obstaculosBase = new Obstaculo[11];

    private Castillo castillo;

    private Enemigo[] enemigos = new Enemigo[5];

    private int enemigosVivos = 0;
    private Enemigo2[] enemigos2 = new Enemigo2[1];

    private int enemigosVivos2 = 0;
    private Escritor t;
    private MostradorDeVida[] vidas = new MostradorDeVida[10];

    private double x;
    private double y;
    private double angulo;
    private double escala;
    private Image imagenFondo;

    private Image imagenFondoTermino = null;

    // Variables y métodos propios de cada grupo
    // ..
    Juego() {
        // Inicializa el objeto entorno
        this.entorno = new Entorno(this, "Super Elizabeth Sis", 800, 600);

        // Inicializar lo que haga falta para el juego
        // ...
        p = new Personaje(140, 300, 20, 50);
        if (!p.isPerdio() && !p.isGano()) {
            Herramientas.play("juego/musica.wav");
        }
    }
}
```

```

        castillo = new Castillo(400, 525, 100, 100);

        imagenFondo = Herramientas.cargarImagen("juego/fondo1.jpg");

        // se crean las vidas
        for (int j = 0; j < p.getVida(); j++) {
            MostradorDeVida v = new MostradorDeVida(j * 50 + 50, 50);
            vidas[j] = v;
        }

        // crea el texto
        t = new Escritor(325, 350, "Winer", "Loser");

        // Inicia el juego!
        this.entorno.iniciar();
    }

    /**
     * Durante el juego, el método tick() será ejecutado en cada instante y por lo
     * tanto es el método más importante de esta clase. Aquí se debe actualizar el
     * estado interno del juego para simular el paso del tiempo (ver el enunciado
     * del TP para mayor detalle).
     */
    public void tick()
    {
        if (imagenFondoTermino == null) {

            entorno.dibujarImagen(imagenFondo, entorno.anch() / 2, entorno.alto()
/ 2, 0, 1);

            for (MostradorDeVida vida : this.vidas) {
                if (vida != null) {

                    vida.dibujar(entorno);
                }
            }

            /* imagen.dibujarImagen(entorno); */
            p.dibujar(entorno);

            /// Valores booleanos usados para los limites
            boolean bloqueaIzquierda = false;
            boolean bloqueaDerecha = false;
            boolean bloqueaAbajo = false;
            boolean bloqueaArriba = false;

            /// Limites de colisión de la linea de los niveles(obstaculos) superiores
            for (Obstaculo obstaculo : obstaculosSuperiores) {
                if (obstaculo != null) {

                    obstaculo.dibujar(entorno);

                    if (p.colisionaPorIzquierda(obstaculo)) {
                        bloqueaIzquierda = true;
                    }
                }
            }
        }
    }

```

```

        if (p.colisionaPorDerecha(obstaculo)) {
            bloqueaDerecha = true;
        }

        if (p.colisionaPorDebajo(obstaculo)) {
            bloqueaAbajo = true;
        }

        if (p.colisionaPorArriba(obstaculo)) {
            bloqueaArriba = true;
        }
    }
}

```

/// Limites de colisión de la linea de niveles inferiores

```

for (Obstaculo o : obstaculosInferiores) {
    if (o != null) {
        o.dibujar(entorno);

        if (p.colisionaPorIzquierda(o)) {
            bloqueaIzquierda = true;
        }

        if (p.colisionaPorDerecha(o)) {
            bloqueaDerecha = true;
        }

        if (p.colisionaPorArriba(o)) {
            bloqueaArriba = true;
        }

        if (p.colisionaPorDebajo(o)) {
            bloqueaAbajo = true;
        }
    }
}

```

// Limites de las lineas de niveles de la base del juego

```

for (Obstaculo obs : obstaculosBase) {
    if (obs != null) {
        obs.dibujar(entorno);
        if (p.colisionaPorArriba(obs)) {
            bloqueaArriba = true;
        }
        if (p.colisionaPorDebajo(obs)) {
            bloqueaAbajo = true;
        }
        if (p.colisionaPorDerecha(obs)) {
            bloqueaDerecha = true;
        }
        if (p.colisionaPorIzquierda(obs)) {
            bloqueaIzquierda = true;
        }
    }
}

```

```

        for (Enemigo2 enemigo2 : enemigos2) {

            if (enemigo2 != null && enemigo2.getDisparo() != null) {
                controlDelProyectilEnemigo(enemigo2, entorno);
            }
        }
        // Generacion de los enemigos de forma aleatoria
        for (int i = 0; i < enemigos.length; i++) {
            if (enemigos[i] == null && enemigosVivos < enemigos.length
&& !p.isGano() && !p.isPerdio()) {
                Random randomEnemigos = new Random();
                int numeroAleatorio = randomEnemigos.nextInt(0, 2);
                int posAleatoria = randomEnemigos.nextInt(550 - 300
+ 1) + 300;

                int posAleatoria2 = randomEnemigos.nextInt(200 - 100
+ 1) + 100;

                if (numeroAleatorio == 0) {
                    Enemigo e = new Enemigo(entorno.ancho() +
posAleatoria2, posAleatoria, 40, 40, "izquierda");

                    enemigos[i] = e;
                } else {
                    Enemigo e = new Enemigo(0 - posAleatoria2,
posAleatoria, 40, 40, "derecha");

                    enemigos[i] = e;
                }
                enemigosVivos++;
            }
        }
        for (int i = 0; i < enemigos2.length; i++) {

            if (enemigos2[i] == null && enemigosVivos2 <
enemigos2.length && !p.isGano() && !p.isPerdio()) {
                Random randomEnemigos = new Random();
                int direccion = randomEnemigos.nextInt(0, 2);
                int posAleatoria = randomEnemigos.nextInt(1000 - 800
+ 1) + 800;

                int posAleatoria2 =
randomEnemigos.nextInt(entorno.ancho() - 0 + 1) + 0;
                if (direccion == 0) {
                    Enemigo2 e = new Enemigo2(posAleatoria2, 0
- posAleatoria, 40, 40, "derecha");

                    enemigos2[i] = e;
                } else {
                    Enemigo2 e = new Enemigo2(posAleatoria2, 0
- posAleatoria, 40, 40, "izquierda");

                    enemigos2[i] = e;
                }
                enemigosVivos2++;
            }
        }
        for (int i = 0; i < enemigos2.length; i++) {

```

```

        Enemigo2 enemigo = enemigos2[i];

        if (enemigo != null && !p.isGano() && !p.isPerdio()) {

            // Colisión con disparo
            if (p.getDisparo() != null &&
                p.getDisparo().colisionaDisparoConEnemigo2(enemigo)) {
                Herramientas.play("juego/explocion.wav");
                enemigos2[i] = null;
                p.setDisparo(null);
                enemigosVivos2--;
                continue;
            }

            // Colisión con barrera
            if (enemigo.colisionaConBarrera(p)) {

                enemigos2[i] = null;
                enemigosVivos2--;
                Herramientas.play("juego/explocion.wav");
                continue;
            }

            // Dibujar enemigo
            enemigo.dibujar(entorno);

            // Colisión con jugador
            if (enemigo.colisionaConElJugador(p)) {

                enemigos2[i] = null;
                enemigosVivos2--;
                vidas = convertirANULLaVida(entorno, vidas,
p);

                continue;
            }

            // Movimiento
            if (enemigo.getY() <= 50) {

                enemigo.moverAbajo();

            } else {

                if (enemigo.getDisparo() == null) {

                    Herramientas.play("juego/disparo.wav");

                    enemigo.disparo(p);

                    enemigo.setDisparoTocoJugador(false);

                }
                if (enemigo.getDisparo() != null) {

                    enemigo.getDisparo().dibujar(entorno);

                    enemigo.getDisparo().mover();

```

```

                                if
(enemigo.disparoColisionaJugador(p)) {
Herramientas.play("juego/explosion.wav");
                                vidas =
convertirANullaVida(entorno, vidas, p);
                                }
                                }
                                if (enemigo.getDireccion().equals("derecha"))
{
                                enemigo.moverDerecha();
                                if
(enemigo.cambiarDireccionDerecha(entorno)) {
enemigo.setDireccion("izquierda");
                                }
                                } else {
                                enemigo.moverIzquierda();
                                if
(enemigo.cambiarDireccionIzquierda(entorno)) {
enemigo.setDireccion("derecha");
                                }
                                }
                                }
                                }
                                }
// while(enemigosVivos<limiteEnemigos) {
//
//
// }
// Dibujo de los enemigos y control de colisiones entre los obstaculos y
el
// jugador
for (int i = 0; i < enemigos.length; i++) {
    Enemigo enemigo = enemigos[i];

    if (enemigo != null) {

        if (p.getDisparo() != null) {
            if
(p.getDisparo().colisionaDisparoConEnemigo(enemigo)) {
                enemigo = null;
                enemigos[i] = enemigo;
                p.setDisparo(null);
                enemigosVivos--;

Herramientas.play("juego/explosion.wav");
            }

        }

        for (Obstaculo o : obstaculosSuperiores) {
            if (enemigo != null && o != null &&
enemigo.colisionaConObstaculo(o)) {

```

```

        enemigo = null;
        enemigos[i] = enemigo;
        enemigosVivos--;
    }
}
for (Obstaculo o1 : obstaculosInferiores) {
    if (enemigo != null && o1 != null &&
enemigo.colisionaConObstaculo(o1)) {
        enemigo = null;
        enemigos[i] = enemigo;
        enemigosVivos--;
    }
}
if (enemigo != null &&
enemigo.colisionaConBarrera(p)) {
    enemigo = null;
    enemigos[i] = enemigo;
    enemigosVivos--;
    Herramientas.play("juego/explocion.wav");
}
if (enemigo != null) {
    enemigo.dibujar(entorno);
    if (enemigo.colisionaConElJugador(p)) {
        enemigo = null;
        enemigos[i] = enemigo;
        enemigosVivos--;
Herramientas.play("juego/explocion.wav");
        vidas =
convertirANUILLaVida(entorno, vidas, p);
    } else {
        if
(enemigo.getDireccion().equals("izquierda")) {
            enemigo.moverIzquierda();
            if
(enemigo.esDestruiblePorIzquierda(entorno)) {
                enemigo = null;
                enemigos[i] =
enemigosVivos--;
            }
        } else {
            enemigo.moverDerecha();
            if
(enemigo.esDestruibleDerecha(entorno)) {
                enemigo = null;
                enemigos[i] =
enemigosVivos--;
            }
        }
    }
}
enemigo;

```

```

    }
    }
    }
    }
    }

    // Repetición de la linea superior de niveles
    if (!p.isPerdio()) {
        detectaElMovimiento(entorno, p);
        int valorY = 400;
        int anchoObstaculo = 120;
        int altoObstaculo = 20;
        for (int i = 0; i < obstaculosSuperiores.length; i++) {
            Random valoresRandoms = new Random();
            int numerosRan = valoresRandoms.nextInt(501 - 200 +
1) + 200;
            if (obstaculosSuperiores[i] == null &&
!p.getEnMovimiento()) {
                Obstaculo o = new Obstaculo(500 * i +
numerosRan, valorY, anchoObstaculo, altoObstaculo);
                obstaculosSuperiores[i] = o;
            }
            if (obstaculosSuperiores[i] != null &&
p.getEnMovimiento()) {
                obstaculosSuperiores[i].setX(obstaculosSuperiores[i].getX() - 2);
            }
            obstaculosSuperiores[i].dibujar(entorno);
        }
    }

    /*
    * for(Obstaculo a: obstaculos) { if(p.getEnMovimiento()) {
a.setX(a.getX()-2);}
    * if(a.bordeDerecho()<0) { Random ran=new Random(); int r=
    * ran.nextInt(entorno.alto()); a.setX(entorno.ancho()+r/2); }
    * }
    */

    // Repetición de la linea inferior de niveles
    detectaElMovimiento(entorno, p);
    int anchoObstaculo1 = 120;
    for (int i = 0; i < obstaculosInferiores.length; i++) {
        int valorY1 = 500;
        Random valoresRandoms = new Random();
        int numerosRan = valoresRandoms.nextInt(401 - 200 +
1) + 200;
        if (obstaculosInferiores[i] == null &&
!p.getEnMovimiento()) {
            Obstaculo o = new Obstaculo(400 * i +
numerosRan, valorY1, anchoObstaculo1, altoObstaculo);
            obstaculosInferiores[i] = o;
        }
    }

```

```

        if (obstaculosInferiores[i] != null &&
p.getEnMovimiento()) {
    obstaculosInferiores[i].setX(obstaculosInferiores[i].getX() - 2);
    }
    obstaculosInferiores[i].dibujar(entorno);
}
// Repetición de la línea de las bases
detectaElMovimiento(entorno, p);
for (int i = 0; i < obstaculosBase.length; i++) {
    if (obstaculosBase[i] == null &&
!p.getEnMovimiento()) {
        Obstaculo o = new Obstaculo(400 * i, 595,
290, 40);
        obstaculosBase[i] = o;
    }
    if (obstaculosBase[i] != null && p.getEnMovimiento())
    {
        obstaculosBase[i].setX(obstaculosBase[i].getX() - 2);
        }
        if (i == obstaculosBase.length - 1) {
            castillo.setX(obstaculosBase[i].getX());
            // castillo.setY(obstaculosBase[i].getY() +
castillo.bordeInferior());
        }
        obstaculosBase[i].dibujarBase(entorno);
    }

    castillo.dibujar(entorno);
    if (p.isSeMueve() == true) {
        castillo.setX(castillo.getX() - 1);
    }
    if (p.isGano()) {
        t.dibujar('W', entorno);
    }
    ;

    // posición del personaje en mitad de pantalla
    if (p.getX() > entorno.anch() / 2) {
        p.setX((entorno.anch() / 2));
    }

    if (!p.isGano() && !p.isPerdio()) {
        controlMovimientoJugador(entorno, p,
bloqueaIzquierda, bloqueaDerecha, bloqueaAbajo, bloqueaArriba);
        controlDelSalto(p, bloqueaArriba);

        controlDelProyectil(p, entorno);

        controlDelBarrera(entorno, p);

        p.colisionaCastillo(castillo);

```

```

    }

    if (p.siElPersonajeTocaElBordeInferiorDeLaPantalla())// aqui
    puse que el personaje se teletransporte

    // caundo se cae
    {
        vidas = convertirANUILLaVida(entorno, vidas, p);
    }
}
if (p.isGano() || p.isPerdio()) {
    Image icono =
Herramientas.cargarImagen("juego/Corazon.jpg");
    this.imagenFondoTermino = icono;
}
}

else {
    entorno.dibujarImagen(imagenFondoTermino, 400, 300, 0, 0.08);
    if (p.isPerdio()) {
        t.dibujar('L', entorno);
    }
    if (p.isGano()) {
        t.dibujar('W', entorno);
    }
    if (entorno.sePresiono('r') || entorno.sePresiono('R')) {
        sumarVida(entorno, vidas, p, -1);
        enemigos = eliminarTodosLosEnemigos(entorno, enemigos);
        enemigosVivos -= enemigos.length;
        p.setX(140);
        p.setY(300);
        imagenFondoTermino = null;
        p.setGano(false);
        p.setPerdio(false);
    }
}
}

public static void detectaElMovimiento(Entorno a, Personaje b) {
    if (a.estaPresionada(a.TECLA_DERECHA) && b.getX() >= 400) { // se agrego
    "b.getX()>=400" para que solo se mueba

    // si esta a mitad de pantalla
    b.setEnMovimiento(true);
    } else {
        b.setEnMovimiento(false);
    }
    if (a.estaPresionada(a.TECLA_DERECHA) &&
a.estaPresionada(a.TECLA_IZQUIERDA)) {
        b.setEnMovimiento(false);
    }
}
/*
* if(c.getEnPantalla()==true){ for(Obstaculo obs: o) { obs.setSeMueven(false);
* } for (Obstaculo e: obstaculos) { e.setSeMueven(false); } }
*/

```

```

    }

    public static void controlDelProyectil(Personaje p, Entorno entorno) {
        if (p.getDisparo() != null) {
            p.getDisparo().dibujar(entorno);
        }

        // movimiento del proyectil
        if (p.getDisparo() != null) {
            p.getDisparo().mover();
        }

        // colisiones del proyectil
        if (p.getDisparo() != null && p.getDisparo().getX() < 0) {
            p.setDisparo(null);
        }

        // colision del lado derecho
        if (p.getDisparo() != null && p.getDisparo().getX() > entorno.ancha()) {
            p.setDisparo(null);
        }

        // colision por la parte superior
        if (p.getDisparo() != null && p.getDisparo().getY() < 0) {
            p.setDisparo(null);
        }

        // colision por la parte inferior
        if (p.getDisparo() != null && p.getDisparo().getY() > entorno.alto()) {
            p.setDisparo(null);
        }
    }

    public static void controlDelProyectilEnemigo(Enemigo2 e, Entorno entorno) {

        // colisiones del proyectil
        if (e.getDisparo() != null && e.getDisparo().getX() < 0) {
            e.setDisparo(null);
        }

        // colision del lado derecho
        if (e.getDisparo() != null && e.getDisparo().getX() > entorno.ancha()) {
            e.setDisparo(null);
        }

        // colision por la parte superior
        if (e.getDisparo() != null && e.getDisparo().getY() < 0) {
            e.setDisparo(null);
        }

        // colision por la parte inferior
        if (e.getDisparo() != null && e.getDisparo().getY() > entorno.alto()) {
            e.setDisparo(null);
        }
    }

```

```

    }

}

public static void controlDelBarrera(Entorno e, Personaje p) {
    if (p.getBarrera() != null) {
        p.getBarrera().dibujar(e);
        p.getBarrera().movimiento(p.getX(), p.getY());
        p.getBarrera().actualizarTiempoDeAparicion(e, e.tiempo());
    }
    if (p.getBarrera() != null && p.getBarrera().getTiempoActual() >=
p.getBarrera().getTiempoDeDesaparicion()) {
        p.setBarrera(null);
    }
}

// funcion que controla las colisiones dado un entorno personaje o obstaculo
public static void controlMovimientoJugador(Entorno entorno, Personaje p, boolean
bloqueaIzquierda,
        boolean bloqueaDerecha, boolean bloqueaAbajo, boolean
bloqueaArriba) {
    if (entorno.sePresiono(entorno.TECLA_ARRIBA) && !bloqueaArriba
        && (bloqueaAbajo || p.bordeInferior() >= entorno.alto())) {
        Herramientas.play("juego/retroJump.wav");
        p.setEstaSaltando(true);
        p.setSaltoY(p.getY() - 200);
    }

    if (entorno.estaPresionada(entorno.TECLA_IZQUIERDA) &&
p.bordeIzquierdo() > 0 && !bloqueaIzquierda) {

        p.moverIzquierda();
    }

    if (entorno.estaPresionada(entorno.TECLA_DERECHA) && p.bordeDerecho()
< entorno.ancho() + 2
        && !bloqueaDerecha) {

        p.moverDerecha();
    }

    if (p.getTieneGravedad() && p.bordeInferior() <= entorno.alto() - 2 &&
!bloqueaAbajo) {

        p.moverAbajo();
    }
    if (entorno.sePresionoBoton(entorno.BOTON_DERECHO) && p.getBarrera()
== null) {

        p.creacionDeBarrera(entorno, entorno.tiempo());
    }
    if (entorno.sePresionoBoton(entorno.BOTON_IZQUIERDO) && p.getDisparo()
== null) {

        Herramientas.play("juego/disparo.wav");
        p.disparo(entorno);
    }
}

```

```

    }

    public static void controlDelSalto(Personaje p, boolean bloqueaArriba) {
        if (p.isEstaSaltando()) {
            if (!bloqueaArriba) {
                p.setTieneGravedad(false);
                p.saltar();

                if (p.getY() <= p.getSaltoY()) {
                    p.setTieneGravedad(true);
                    p.setEstaSaltando(false);
                }
            } else {
                p.setTieneGravedad(true);
                p.setEstaSaltando(false);
            }
        }
    }
}

public static MostradorDeVida[] convertirANUILLaVida(Entorno e, MostradorDeVida[] v,
Personaje p) {
    boolean yaSeQuito = false;
    for (int i = v.length - 1; i > -1; i = i - 1) {
        if (v[i] != null && yaSeQuito == false) {
            yaSeQuito = true;
            v[i] = null;
        }
        if (v[0] == null) {
            Herramientas.play("juego/lose.wav");
            p.setPerdio(true);
        }
    }
    return v;
}

public static MostradorDeVida[] sumarVida(Entorno e, MostradorDeVida[] v, Personaje
p, int vidaSumada) {
    boolean yaSeAgrego = false;
    if (vidaSumada == -1) {
        for (int i = 0; i < p.getVida(); i++) {
            MostradorDeVida vida = new MostradorDeVida(i * 50 + 50,
50);
            v[i] = vida;
        }
    }

    if (vidaSumada > 0) {
        for (int i = 0; i < p.getVida(); i++) {
            if (v[i] == null) {
                MostradorDeVida vida = new MostradorDeVida(i * 50
+ 50, 50);
                v[i] = vida;
            }
        }
    }
}

```

```

        }

        return v;
    }

    public static Enemigo[] eliminarTodosLosEnemigos(Entorno e, Enemigo[] enemigos) {
        for (int i = 0; i < enemigos.length; i++) {
            if (enemigos[i] != null) {
                enemigos[i] = null;
            }
        }
        return enemigos;
    }

    @SuppressWarnings("unused")
    public static void main(String[] args) {
        Juego juego = new Juego();
    }
}

```

Clase “Barrera.java”

```

package juego;

import java.awt.Color;

import entorno.Entorno;

public class Barrera {
    private double x;
    private double y;
    private double ancho;
    private double distancia;
    private double tiempoActual;
    private double tiempoDeDesaparicion;

    public Barrera(double x, double y, double ancho, double distancia, int tiempo) {
        super();
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.distancia = distancia;
        this.tiempoActual = tiempo;
        this.tiempoDeDesaparicion = tiempo + 1000;
    }

    public void dibujar(Entorno e) {
        actualizarTiempoDeAparicion(e, tiempoActual);
        e.dibujarRectangulo(x + 20, y, ancho, distancia, 0, Color.RED);
    }

    public void actualizarTiempoDeAparicion(Entorno e, double tiempo) {
        this.tiempoActual = tiempo;
    }
}

```

```
public void movimiento(int xDePersonaje, int yDePersonaje) {
    this.x = xDePersonaje;
    this.y = yDePersonaje;
}

public double bordeDerecho() {
    return this.x + this.anchos / 2;
}

public double bordeIzquierdo() {
    return this.x - this.anchos / 2;
}

public double bordeInferior() {
    return this.y + this.distancia / 2;
}

public double bordeSuperior() {
    return this.y - this.distancia / 2;
}

public double getX() {
    return x;
}

public void setX(double x) {
    this.x = x;
}

public double getY() {
    return y;
}

public void setY(double y) {
    this.y = y;
}

public double getAncho() {
    return anchos;
}

public void setAncho(double ancho) {
    this.anchos = ancho;
}

public double getDistancia() {
    return distancia;
}

public void setDistancia(double distancia) {
    this.distancia = distancia;
}

public double getTiempoActual() {
    return tiempoActual;
}
```

```

        public void setTiempoActual(double tiempoActual) {
            this.tiempoActual = tiempoActual;
        }

        public double getTiempoDeDesaparicion() {
            return tiempoDeDesaparicion;
        }

        public void setTiempoDeDesaparicion(double tiempoDeDesaparicion) {
            this.tiempoDeDesaparicion = tiempoDeDesaparicion;
        }
    }
}

```

Clase “Castillo.java”

```

package juego;

import java.awt.Color;

import entorno.Entorno;

public class Castillo {
    private int x;
    private int y;
    private int ancho;
    private int alto;

    /* private boolean enPantalla; */
    public Castillo(int x, int y, int ancho, int alto) {

        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        /* this.enPantalla=false; */
    }

    public void dibujar(Entorno e) {
        e.dibujarRectangulo(x, y, ancho, alto, 0, Color.YELLOW);
    }

    public int getX() {
        return x;
    }

    public void setX(int x) {
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }
}

```

```

    }

    public int getAncho() {
        return ancho;
    }

    public void setAncho(int ancho) {
        this.ancho = ancho;
    }

    public int getAlto() {
        return alto;
    }

    public void setAlto(int alto) {
        this.alto = alto;
    }

    public int bordeDerecho() {
        return this.x + this.ancho / 2;
    }

    public int bordeIzquierdo() {
        return this.x - this.ancho / 2;
    }

    public int bordeSuperior() {
        return this.y - this.alto / 2;
    }

    public int bordeInferior() {
        return this.y + this.alto / 2;
    }
}
/*
 * public boolean getEnPantalla() { return this.enPantalla; } public void
 * setEnPantalla(boolean a) { this.enPantalla=a; }
 */
}

```

Clase “Castillo.java”

```

package juego;

import java.awt.Color;

import entorno.Entorno;

public class Castillo {
    private int x;
    private int y;
    private int ancho;
    private int alto;

    /* private boolean enPantalla; */
    public Castillo(int x, int y, int ancho, int alto) {

```

```

        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        /* this.enPantalla=false; */
    }

    public void dibujar(Entorno e) {
        e.dibujarRectangulo(x, y, ancho, alto, 0, Color.YELLOW);
    }

    public int getX() {
        return x;
    }

    public void setX(int x) {
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }

    public int getAncho() {
        return ancho;
    }

    public void setAncho(int ancho) {
        this.ancho = ancho;
    }

    public int getAlto() {
        return alto;
    }

    public void setAlto(int alto) {
        this.alto = alto;
    }

    public int bordeDerecho() {
        return this.x + this.ancho / 2;
    }

    public int bordeIzquierdo() {
        return this.x - this.ancho / 2;
    }

    public int bordeSuperior() {
        return this.y - this.alto / 2;
    }

    public int bordeInferior() {

```

```

        return this.y + this.alto / 2;
    }
    /*
    * public boolean getEnPantalla() { return this.enPantalla; } public void
    * setEnPantalla(boolean a) { this.enPantalla=a; }
    */
}

```

Clase “Enemigo.java”

```

package juego;

import java.awt.Color;
import java.awt.Image;

import entorno.Entorno;
import entorno.Herramientas;

public class Enemigo {
    private int x;
    private int y;
    private int ancho;
    private int alto;
    private int velocidadX = 3;
    private String direccion;
    /*
    * private Image imagenParado; private boolean adelante; private Image
    * imagenAdelante; private boolean atras; private Image imagenAtras;
    */

    public Enemigo(int x, int y, int ancho, int alto, String direccion) {

        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        this.direccion = direccion;
        /*
        *      Image      icono=      Herramientas.cargarImagen("juego/2.jpeg");
this.imagenAdelante =
        * icono; Image icono2= Herramientas.cargarImagen("juego/1.jpeg");
        * this.imagenAtras = icono2; Image icono3=
        * Herramientas.cargarImagen("juego/3.jpeg"); this.imagenParado = icono3;
        */

    }

    public String getDireccion() {
        return direccion;
    }

    public void setDireccion(String direccion) {
        this.direccion = direccion;
    }
}

```

```

public void dibujar(Entorno e) {
    e.dibujarRectangulo(x, y, ancho, alto, 0, Color.BLUE);
}

public void moverIzquierda() {
    this.x = this.x - velocidadX;
}

public void moverDerecha() {
    this.x = this.x + velocidadX;
}

public boolean esDestruibleDerecha(Entorno e) {
    if (bordeIzquierdo() >= e.ancho()) {

        return true;

    }
    return false;
}

public boolean esDestruiblePorIzquierda(Entorno e) {
    if (bordeDerecho() <= 0) {

        return true;

    }
    return false;
}

public int bordeIzquierdo() {
    return this.x - this.ancho / 2;
}

public int bordeDerecho() {
    return this.x + this.ancho / 2;
}

public int bordeSuperior() {
    return this.y - this.alto / 2;
}

public int bordeInferior() {
    return this.y + this.alto / 2;
}

public boolean colisionaConElJugador(Personaje p) {

    if (bordeIzquierdo() <= p.bordeDerecho() && bordeDerecho() >= (p.getX() &&
bordeSuperior() <= p.bordeInferior()
        && bordeInferior() >= p.bordeSuperior()) {
        return true;
    }
    return false;
}

```

```

    }

    public boolean colisionaConObstaculo(Obstaculo o) {

        if (bordeInferior() >= o.bordeSuperior() && bordeSuperior() <=
o.bordeInferior()) {

            return true;
        }
        return false;
    }

    public boolean colisionaConBarrera(Personaje p) {
        if (p.getBarrera() != null) {
            if (bordeIzquierdo() <= p.getBarrera().bordeDerecho() + 20
&& bordeDerecho() >= p.getBarrera().bordeDerecho())
{
                if (bordeInferior() >= p.getBarrera().bordeSuperior()
&& bordeSuperior() <=
p.getBarrera().bordeInferior()) {
                    return true;
                }
            }
        }
        return false;
    }

    public int getX() {
        return x;
    }

    public void setX(int x) {
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }

    public int getAncho() {
        return ancho;
    }

    public void setAncho(int ancho) {
        this.ancho = ancho;
    }

    public int getAlto() {
        return alto;
    }

    public void setAlto(int alto) {
        this.alto = alto;
    }

```

```

    }

    public int getVelocidadX() {
        return velocidadX;
    }

    public void setVelocidadX(int velocidadX) {
        this.velocidadX = velocidadX;
    }
}

```

Clase “Enemigo2.java”

```

package juego;

import java.awt.Color;
import java.awt.Image;

import entorno.Entorno;
import entorno.Herramientas;

public class Enemigo2 {
    private int x;
    private int y;
    private int ancho;
    private int alto;
    private int velocidadX = 2;
    private int velocidadY = 3;
    private String direccion;
    private double aceleracion = 0.5;
    private Proyectil disparo;
    private boolean disparoTocoJugador = false;
    private Image imagen;

    public Enemigo2(int x, int y, int ancho, int alto, String direccion) {

        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        this.direccion = direccion;
        this.disparo = null;
        this.imagen = Herramientas.cargarImagen("juego/enemigoVolador.png");

    }

    public String getDireccion() {
        return direccion;
    }

    public void setDireccion(String direccion) {
        this.direccion = direccion;
    }

    public void dibujar(Entorno e) {

```

```

        e.dibujarRectangulo(x, y, ancho, alto, 0, Color.BLUE);
        e.dibujarImagen(imagen, x, y, 0, 0.19);
    }

    public void moverIzquierda() {
        this.x = this.x - velocidadX;
    }

    public void moverDerecha() {
        this.x = this.x + velocidadX;
    }

    public void moverAbajo() {
        velocidadY += aceleracion;
        y += velocidadY;
    }

    public boolean cambiarDireccionDerecha(Entorno e) {
        if (bordeDerecho() >= e.ancho()) {

            return true;
        }

        return false;
    }

    public boolean cambiarDireccionIzquierda(Entorno e) {
        if (bordeIzquierdo() <= 0) {

            return true;
        }

        return false;
    }

    public int bordeIzquierdo() {
        return this.x - this.ancho / 2;
    }

    public int bordeDerecho() {
        return this.x + this.ancho / 2;
    }

    public int bordeSuperior() {
        return this.y - this.alto / 2;
    }

    public int bordeInferior() {
        return this.y + this.alto / 2;
    }

    public boolean colisionaConElJugador(Personaje p) {

        if (bordeIzquierdo() <= p.bordeDerecho() && bordeDerecho() >= (p.getX()) &&
bordeSuperior() <= p.bordeInferior()
            && bordeInferior() >= p.bordeSuperior()) {

```

```

        return true;
    }
    return false;
}

public boolean colisionaConObstaculo(Obstaculo o) {

    if (bordeInferior() >= o.bordeSuperior() && bordeSuperior() <=
o.bordeInferior()) {

        return true;
    }
    return false;
}

public boolean colisionaConBarrera(Personaje p) {
    if (p.getBarrera() != null) {
        if (bordeIzquierdo() <= p.getBarrera().bordeDerecho() + 20
            && bordeDerecho() >= p.getBarrera().bordeDerecho())
        {
            if (bordeInferior() >= p.getBarrera().bordeSuperior()
                && bordeSuperior() <=
p.getBarrera().bordeInferior()) {
                return true;
            }
        }
    }
    return false;
}

public boolean disparoColisionaJugador(Personaje p) {

    if (this.disparo != null && !disparoTocoJugador) {
        if (this.disparo.bordeDerecho() >= p.bordeIzquierdo() &&
this.disparo.bordeSuperior() <= p.bordeInferior()
            && this.disparo.bordeInferior() >= p.bordeSuperior()
            && this.disparo.bordeIzquierdo() <=
p.bordeDerecho()) {
            this.disparoTocoJugador = true;
            return true;
        }
    }
    return false;
}

public boolean isDisparoTocoJugador() {
    return disparoTocoJugador;
}

public void setDisparoTocoJugador(boolean disparoTocoJugador) {
    this.disparoTocoJugador = disparoTocoJugador;
}

public int getX() {
    return x;
}

```

```

    public void setX(int x) {
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }

    public int getAncho() {
        return ancho;
    }

    public void setAncho(int ancho) {
        this.ancho = ancho;
    }

    public int getAlto() {
        return alto;
    }

    public void setAlto(int alto) {
        this.alto = alto;
    }

    public int getVelocidadX() {
        return velocidadX;
    }

    public void setVelocidadX(int velocidadX) {
        this.velocidadX = velocidadX;
    }

    public void disparo(Personaje p) {
        int xMouse = p.getX();
        int yMouse = p.getY();

        this.disparo = new Proyectil(this.x, this.y, 20, xMouse, yMouse);
    }

    public Proyectil getDisparo() {
        return this.disparo;
    }

    public void setDisparo(Proyectil disparo) {
        this.disparo = disparo;
    }
}

```

Clase “Escrito.java”

```
package juego;
```

```

import java.awt.Color;

import entorno.Entorno;

public class Escritor {
    private int x;
    private int y;
    private String textoGanador;
    private String textoPerdedor;

    public Escritor(int x, int y, String textoGanador, String textoPerdedor) {
        super();
        this.x = x;
        this.y = y;
        this.textoGanador = textoGanador;
        this.textoPerdedor = textoPerdedor;
    }

    public void dibujar(char c, Entorno e) {
        e.cambiarFont("Arial", 50, Color.white);
        if (c == 'W') {
            e.escribirTexto(textoGanador, x, y);
        }
        if (c == 'L') {
            e.escribirTexto(textoPerdedor, x, y);
        }
    }

    public int getX() {
        return x;
    }

    public void setX(int x) {
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }

    public String getTextoGanador() {
        return textoGanador;
    }

    public void setTextoGanador(String textoGanador) {
        this.textoGanador = textoGanador;
    }

    public String getTestoPerdedor() {
        return textoPerdedor;
    }
}

```

```
        public void setTestoPerdedor(String testoPerdedor) {
            this.textoPerdedor = testoPerdedor;
        }
    }
}
```

Clase “MostradorDeVida.java”

```
package juego;

import javax.swing.*;

import entorno.Entorno;
import entorno.Herramientas;

import java.awt.*;

public class MostradorDeVida {
    private int x;
    private int y;
    private Image imagen;

    public MostradorDeVida(int x, int y) {
        this.x = x;
        this.y = y;
        Image icono = Herramientas.cargarImagen("juego/Corazon.jpg");
        this.imagen = icono;
    }

    public void dibujar(Entorno e) {
        e.dibujarImagen(imagen, x, y, 0, 0.08);
    }

    public int getX() {
        return x;
    }

    public void setX(int x) {
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }
}
```

Clase “Obstaculo.java”

```
package juego;

import java.awt.Color;
import java.awt.Image;
import entorno.Entorno;
import entorno.Herramientas;

public class Obstaculo {
    private int x;
    private int y;
    private int ancho;
    private int alto;
    private Image imagen;

    public Obstaculo(int x, int y, int ancho, int alto) {
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        this.imagen = Herramientas.cargarImagen("juego/baseVerde1.png");
    }

    public void dibujar(Entorno e) {

        e.dibujarImagen(imagen, x, y, 0, 0.190);
    }

    public void dibujarBase(Entorno e) {

        e.dibujarImagen(imagen, x, y, 0, 0.43);
    }

    public int bordeDerecho() {

        return this.x + this.ancho / 2;
    }

    public int bordeIzquierdo() {

        return this.x - this.ancho / 2;
    }

    public int bordeInferior() {

        return this.y + this.alto / 2;
    }

    public int bordeSuperior() {

        return this.y - this.alto / 2;
    }

    public int getX() {

        return x;
    }

    public void setX(int x) {
```

```

        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }

    public int getAncho() {
        return ancho;
    }

    public void setAncho(int ancho) {
        this.ancho = ancho;
    }

    public int getAlto() {
        return alto;
    }

    public void setAlto(int alto) {
        this.alto = alto;
    }

    public int setBordeDerecho(int a) {
        return (this.x + this.ancho / 2) + a;
    }

    public int setBordeIzquierdo(int a) {
        return (this.x + this.ancho / 2) + a;
    }
}
/*
 * public boolean getSeMueven() { return this.seMueven; } public void
 * setSeMueven(boolean a) { this.seMueven=a; }
 */
}

```

Clase “Personaje.java”

```

package juego;

import java.awt.Color;
import java.awt.Image;

import entorno.Entorno;
import entorno.Herramientas;

public class Personaje {
    private int x;
    private int y;
    private int ancho;
    private int alto;
}

```

```

private int saltoY;
private Proyectil disparo;
private boolean tieneGravedad;
private boolean estaSaltando;
private boolean seMueve;
private int velocidadX = 5;
private int velocidadY = 5;
private int vida = 10;
private boolean llegoAlCastillo;
private boolean gano;
private boolean perdio;
private Barrera barrera;
private Image imagen;

public Personaje(int x, int y, int ancho, int alto) {
    this.x = x;
    this.y = y;
    this.saltoY = 0;
    this.ancho = ancho;
    this.alto = alto;
    this.disparo = null;
    this.barrera = null;
    this.tieneGravedad = true;
    this.estaSaltando = false;
    this.gano = false;
    this.perdio = false;
    this.seMueve = false;
    this.llegoAlCastillo = false;
    this.imagen = Herramientas.cargarImagen("juego/personaje1.png");
}

public boolean getEnMovimiento() {
    return seMueve;
}

public void setEnMovimiento(boolean a) {
    this.seMueve = a;
}

public boolean isEstaSaltando() {
    return estaSaltando;
}

public void setEstaSaltando(boolean estaSaltando) {
    this.estaSaltando = estaSaltando;
}

public boolean getTieneGravedad() {
    return tieneGravedad;
}

public void setTieneGravedad(boolean tieneGravedad) {
    this.tieneGravedad = tieneGravedad;
}

public int getSaltoY() {

```

```

        return saltoY;
    }

    public void setSaltoY(int saltoY) {
        this.saltoY = saltoY;
    }

    public void dibujar(Entorno e) {
        e.dibujarImagen(imagen, x, y, 0, 0.17);
    }

    public void moverIzquierda() {
        this.x = this.x - velocidadX;
    }

    public void moverDerecha() {
        this.x = this.x + velocidadX;
    }

    public void moverArriba() {
        this.y = this.y - velocidadY;
    }

    public void saltar() {
        this.y = this.y - velocidadY;
    }

    public void moverAbajo() {
        this.y = this.y + velocidadY;
    }

    public boolean colisionaPorIzquierda(Obstaculo o) {
        if (bordeIzquierdo() <= o.bordeDerecho() && bordeDerecho() >= (o.getX())) {
            if (bordeInferior() >= o.bordeSuperior() + 1 && bordeSuperior() <=
o.bordeInferior() - 1) {
                rebote(+5);
                System.out.println("+5");
                return true;
            }
        }
        return false;
    }

    public boolean colisionaPorDerecha(Obstaculo o) {
        if (bordeDerecho() >= o.bordeIzquierdo() && bordeIzquierdo() <= o.getX()) {
            if (bordeInferior() >= o.bordeSuperior() + 1 && bordeSuperior() <=
o.bordeInferior() - 1) {
                rebote(-5);
                System.out.println("-5");
                return true;
            }
        }
    }

```

```

        return false;
    }

    // aqui se agregaron mas coliciones, especificamente si coliciona por debajo o
    // por arriba
    public boolean colisionaPorArriba(Obstaculo o) {
        if (bordeSuperior() <= o.bordeInferior() && bordeInferior() >=
o.bordeSuperior() + 1) {
            if (bordeDerecho() >= o.bordeIzquierdo() + 1 && bordeIzquierdo() <=
o.bordeDerecho() - 1) {
                return true;
            }
        }
        return false;
    }

    public boolean colisionaPorDebajo(Obstaculo o) {
        if (bordeInferior() >= o.bordeSuperior() && bordeSuperior() <=
o.bordeInferior() - 1) {
            if (bordeIzquierdo() <= o.bordeDerecho() - 1 && bordeDerecho() >=
o.bordeIzquierdo() + 1) {
                return true;
            }
        }
        return false;
    }

    public void colisionaCastillo(Castillo c) {
        if (bordeIzquierdo() <= c.bordeDerecho() && bordeDerecho() >=
(c.bordeIzquierdo())) {
            if (bordeInferior() >= c.bordeSuperior() + 1 && bordeSuperior() <=
c.bordeInferior() - 1) {

                this.gano = true;
            }
        }
    }

    public boolean isGano() {
        return this.gano;
    }

    public void setGano(boolean gano) {
        this.gano = gano;
    }

    // aqui se agrego el rebote de el personaje para que no se quede atrapado
    public void rebote(int a) {
        this.x += a;
    }

    // aqui se agrego para teletransportar en caso de que el personaje toque el

```

```

// borde inferior de la pantalla
public boolean siElPersonajeTocaElBordeInferiorDeLaPantalla() {
    if (this.y >= 575) {
        this.y = 100;
        this.x = 300;
        return true;
    }

    return false;
}

// disparo
public void disparo(Entorno e) {
    int xMouse = e.mouseX();
    int yMouse = e.mouseY();

    this.disparo = new Proyectil(this.x, this.y, 30, xMouse, yMouse);
}

public int bordeDerecho() {
    return this.x + this.anchos / 2;
}

public int bordeIzquierdo() {
    return this.x - this.anchos / 2;
}

public int bordeInferior() {
    return this.y + this.alto / 2;
}

public int bordeSuperior() {
    return this.y - this.alto / 2;
}

public void creacionDeBarrera(Entorno e, int tiempo) {
    this.barrera = new Barrera(this.x, this.y, 10, 100, tiempo);
}

public int getX() {
    return x;
}

public void setX(int x) {
    this.x = x;
}

public int getY() {
    return y;
}

public void setY(int y) {
    this.y = y;
}

public int getAncho() {

```

```
        return ancho;
    }

    public void setAncho(int ancho) {
        this.ancho = ancho;
    }

    public int getAlto() {
        return alto;
    }

    public void setAlto(int alto) {
        this.alto = alto;
    }

    public Proyectil getDisparo() {
        return this.disparo;
    }

    public void setDisparo(Proyectil disparo) {
        this.disparo = disparo;
    }

    public int getVida() {
        return vida;
    }

    public boolean getLlegoCastillo() {
        return this.llegoAlCastillo;
    }

    public void setLlegoCastillo(boolean v) {
        this.llegoAlCastillo = v;
    }

    public void setSeMueve(boolean seMueve) {
        this.seMueve = seMueve;
    }

    public boolean isPerdio() {
        return perdio;
    }

    public void setPerdio(boolean perdio) {
        this.perdio = perdio;
    }

    public Barrera getBarrera() {
        return barrera;
    }

    public void setBarrera(Barrera barrera) {
        this.barrera = barrera;
    }

    public boolean isSeMueve() {
        return seMueve;
    }
```

```
}  
}
```

Clase “Proyectil.java”

```
package juego;  
  
import java.awt.Color;  
  
import entorno.Entorno;  
  
public class Proyectil {  
    private double x;  
    private double y;  
    private double diametro;  
    private double dx;  
    private double dy;  
  
    private int velocidad;  
  
    public Proyectil(double x, double y, double diametro, int xMouse, int yMouse) {  
        this.x = x;  
        this.y = y;  
        this.diametro = diametro;  
  
        this.dx = xMouse - this.x;  
        this.dy = yMouse - this.y;  
        this.velocidad = 3;  
        double distancia = Math.sqrt(dx * dx + dy * dy);  
        this.dx = this.dx / (distancia) * velocidad;  
        this.dy = this.dy / (distancia) * velocidad;  
    }  
  
    public double getDx() {  
        return dx;  
    }  
  
    public void setDx(double dx) {  
        this.dx = dx;  
    }  
  
    public double getDy() {  
        return dy;  
    }  
  
    public void setDy(double dy) {  
        this.dy = dy;  
    }  
  
    public void dibujar(Entorno e) {  
        e.dibujarCirculo(x, y, diametro, Color.blue);  
    }  
  
    public void mover() {  
        this.x += this.dx;  
        this.y += this.dy;  
    }  
}
```

```

    }

    public int getVelocidad() {
        return velocidad;
    }

    public void setVelocidad(int velocidad) {
        this.velocidad = velocidad;
    }

    public boolean colisionaDisparoConEnemigo(Enemigo e) {
        if (bordeDerecho() >= e.bordeIzquierdo() && bordeIzquierdo() <=
e.bordeDerecho()
            && bordeInferior() >= e.bordeSuperior() && bordeSuperior()
<= e.bordeInferior()) {
            return true;
        }
        return false;
    }

    public boolean colisionaDisparoConEnemigo2(Enemigo2 e) {
        if (bordeDerecho() >= e.bordeIzquierdo() && bordeIzquierdo() <=
e.bordeDerecho()
            && bordeInferior() >= e.bordeSuperior() && bordeSuperior()
<= e.bordeInferior()) {
            return true;
        }
        return false;
    }

//    public boolean colisionaPorDerecha(Obstaculo o) {
//        if(bordeDerecho()>= o.bordeIzquierdo()&& bordeIzquierdo()<= o.getX()) {
//            if(bordeInferior()>=o.bordeSuperior()+1
bordeSuperior()<=o.bordeInferior()-1) {
//                rebote(-5);
//                System.out.println("-5");
//                return true;
//            }
//        }
//
//        return false;
//    }
//
//    // aqui se agregaron mas coliciones, especificamente si coliciona por debajo o por arriba
//    public boolean colisionaPorArriba(Obstaculo o) {
//        if(bordeSuperior()<=
o.bordeInferior()&&
bordeInferior()>=

```

```

o.bordeSuperior()+1) {
//          if(bordeDerecho())>=o.bordeIzquierdo()+1          &&
bordeIzquierdo())<=o.bordeDerecho()-1) {
//          return true;
//      }
//  }
//      return false;
//  }
//
//  public boolean colisionaPorDebajo(Obstaculo o) {
//      if(bordeInferior())>= o.bordeSuperior()&& bordeSuperior())<= o.bordeInferior()-1) {
//          if(bordeIzquierdo())<=o.bordeDerecho()-1          &&
bordeDerecho())>=o.bordeIzquierdo()+1) {
//          return true;
//      }
//  }
//      return false;
//  }

    public double getX() {
        return x;
    }

    public void setX(double x) {
        this.x = x;
    }

    public double getY() {
        return y;
    }

    public void setY(double y) {
        this.y = y;
    }

    public double getRadio() {
        return diametro;
    }

    public void setRadio(double diametro) {
        this.diametro = diametro;
    }

    public double bordeDerecho() {
        return this.x + this.diametro / 2;
    }

    public double bordeIzquierdo() {
        return this.x - this.diametro / 2;
    }

    public double bordeInferior() {
        return this.y + this.diametro / 2;
    }

```

```
        public double bordeSuperior() {  
            return this.y - this.diametro / 2;  
        }  
    }
```

Conclusion

Que fue una experiencia desafiante, que pudimos aprender muchas cosas, entre ellas como funciona la logica de un videojuego, tambien se supo como trabajar en un grupo de forma organizada y profesional a traves de la herramienta github, este tipo de experiencias nos hace prepararnos para el futuro en un entorno profesional